

Codex GPT-5.4 Reference vs Claude Code Gemma4 12B Evaluation

Generated: 2026-06-19 15:26:29 UTC

Goal

This report evaluates the Claude Code benchmark answer run in ``docs/evaluations/answers/claude_code_gemma4_12b_20260619T120051Z.json`` against the Codex GPT-5.4 reference answers stored in ``docs/evaluations/eval_answers_v2.json``. It explains, question by question, whether the Claude Code answer agrees with the reference answer, partially covers it, or misses the core expected answer.

Test Environment

Codex Reference Side

- Model: ``GPT-5.4 (Codex)``
- Reference file: ``docs/evaluations/eval_answers_v2.json``
- Question source: ``eval_questions_v2.json``
- Evaluation basis: semantic comparison against the Codex reference answer for each question

Claude Code / Ollama Side

- Claude Code model id: ``claude-gemma4-12b``
- Run label: ``gemma4_12b``
- Ollama tag: ``gemma4:12b``
- Host: ``merlin-g-100.psi.ch``
- Anthropic-compatible gateway: ``http://127.0.0.1:4000``
- Allowed Claude Code tools: ``Read,Grep,Glob``
- Run started: ``2026-06-19T12:00:51.804089+00:00``
- Run finished: ``2026-06-19T14:34:52.535662+00:00``
- Recorded answer count: ``100``
- Recorded failure count in metadata: ``6``

Important Caveat

The verdicts are a structured comparison against the Codex reference answers, not a fresh manual audit of every IPPL source file. Correct means the Claude Code answer captures the central reference facts; Partial means it contains relevant signal but misses important scope, exact location, or implementation detail; Incorrect means it failed, abstained, pointed to the wrong subsystem, or omitted the core reference answer.

Because Claude Code answers are often much longer than the concise reference answers, the comparison emphasizes central technical nouns, exact identifiers, files, classes, functions, and implementation steps rather than exact wording.

Overall Result

Metric Value
--- ---:
Questions 100
Correct 53
Partial 27
Incorrect 20
Average score (<code>`Correct=1`, <code>`Partial=0.5`, <code>`Incorrect=0`</code></code>) 0.665 </code>
Runtime/API failures counted as incorrect 4
Answers with leaked self-correction/planning text 5
Mean answer latency 92.40s
Median answer latency 47.86s

Main Findings

- Strongest areas: Class Responsibility, Testing And Workflow. These categories mostly reward direct repository navigation, API identification, and concise file/class role descriptions.
- Weakest areas: Data Flow, Boundary And Halo. These categories were hurt by failed generations, missing exact implementation locations, or cross-file reasoning gaps.
- The run produced ``4`` failed answers that exceeded Claude Code output limits; those failures directly lower the score, especially in data-flow and definition-location questions.
- The Claude Code/Ollama setup was much slower than the earlier direct RAG answer runs: mean latency was ``92.40s`` per question, with a median of ``47.86s``.
- Compared with the direct Gemma4 12B RAG evaluation, this report measures a different answering architecture: Claude Code was allowed to inspect files with ``Read``, ``Grep``, and ``Glob``, then answer the same benchmark questions.

Category Breakdown

Category	Questions	Correct	Partial	Incorrect	Avg score
File Purpose	13	9	3	1	0.808
Definition Location	15	7	2	6	0.533
Class Responsibility	13	12	1	0	0.962
Algorithm	12	5	4	3	0.583
Data Flow	9	1	4	4	0.333
Api Usage	12	7	3	2	0.708
Parallelism And Kokkos	9	5	2	2	0.667
Boundary And Halo	5	1	3	1	0.500
Numerical Meaning	3	1	1	1	0.500
Build And Install	5	2	3	0	0.700
Testing And Workflow	4	3	1	0	0.875

Detailed Per-Question Evaluation

File Purpose

file_001 - What does Ippl.h do?

- Verdict: ****Correct****
- Correct reference answer: Declares the top-level IPPL runtime handles (``Comm``, ``Env``, ``Info``, ``Warn``, ``Error``) and the lifecycle wrappers ``initialize``, ``finalize``, ``fence``, and ``abort``.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.71``
- Reference identifier/location hits: ``9/9``
- Latency: ``67.64s``
- Matched reference signals: ``comm, env, info, warn, error, lifecycle, initialize, finalize``
- Missing/weak reference signals: ``top-level, runtime, handles, wrappers``
- Claude Code answer excerpt: ``Ippl.h` defines the core `ippl` namespace and provides access to global communication and utility objects for the environment. Specifically, it declares: **Communication infrastructure**: Global pointers for a parallel communicator (`Comm`) and an execution environment (`Env`), both managed as `std::unique_ptr`....``

file_002 - What is the purpose of Ippl.cpp?

- Verdict: ****Partial****
- Correct reference answer: Implements runtime setup/teardown: command-line option parsing, MPI communicator/environment creation, logger setup, Kokkos init/finalize, and abort/fence helpers.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.33``
- Latency: ``55.10s``
- Matched reference signals: ``command-line, parsing, mpi, kokkos, helpers``
- Missing/weak reference signals: ``implements, runtime, setup/teardown, option, communicator/environment, creation, logger, setup``
- Claude Code answer excerpt: ``The purpose of `Ippl.cpp` (located in `data/raw/ippl/src/`) is to initialize and finalize the core environment for the **Ippl** framework. Specifically, it performs the following functions within the `ippl` namespace: 1. **Environment Initialization (`initialize`):** * Sets up the MPI environment and communicator (l...``

file_003 - What does FFT.hpp implement?

- Verdict: ****Partial****
- Correct reference answer: Implements ``FFTBase`` setup plus the FFT specialization family for complex/real, sine, cosine, and cosine-I transforms on IPPL fields via `heFFTe`.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.38``
- Reference identifier/location hits: ``0/1``
- Latency: ``44.28s``
- Matched reference signals: ``implements, fft, transforms, fields, heffte``
- Missing/weak reference signals: ``fftbases, setup, specialization, family, complex/real, sine, cosine, cosine-i``
- Claude Code answer excerpt: ``FFT.hpp` implements the `FFT` class (specifically as a template within the `ippl` namespace). According to the file's header comments and implementation, it serves as an interface between **IPPL** and **heffte**, performing complex-to-complex and real-to-complex transforms on IPPL Fields. Key details of its impleme...``

file_004 - What is BareField.h responsible for?

- Verdict: ****Correct****

- Correct reference answer: Defines the core distributed field container: Kokkos-backed storage, owned/allocated domains, field layout access, halo exchange hooks, and expression-style assignment.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.56`
- Latency: `31.72s`
- Matched reference signals: `core, field, container, storage, layout, access, halo, assignment`
- Missing/weak reference signals: `distributed, kokkos-backed, owned/allocated, domains, exchange, hooks, expression-style`
- Claude Code answer excerpt: `Based on the content of `data/raw/ippl/src/Field/BareField.h`, **BareField.h** defines the `BareField` class, which is responsible for representing a real field (e.g., physical quantities in a simulation) within the IPPL framework. Key responsibilities and features of `BareField` include: - **Data Storage**: It ma...

file_005 - What does Field.h add on top of BareField?

- Verdict: **Correct**
- Correct reference answer: Adds mesh awareness and boundary-condition management on top of `BareField`, plus volume integral/average helpers.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.38`
- Reference identifier/location hits: `1/1`
- Latency: `49.61s`
- Matched reference signals: `mesh, barefield, volume, BareField`
- Missing/weak reference signals: `awareness, boundary-condition, management, integral/average, helpers`
- Claude Code answer excerpt: `Based on the source code in `data/raw/ippl/src/Field/Field.h`, the `Field` class inherits from `BareField` and adds several components to it: 1. **A Mesh**: It includes a pointer to a mesh object (`mesh_m`) of type `Mesh_t`. 2. **Boundary Conditions (BCs)**: It incorporates boundary conditions via the `bc_m` member,...

file_006 - What role does HaloCells.h play in IPPL?

- Verdict: **Correct**
- Correct reference answer: Defines the halo-exchange machinery for fields: pack/unpack, boundary exchange, and periodic handling for ghost cells.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.60`
- Latency: `27.57s`
- Matched reference signals: `boundary, exchange, periodic, handling, ghost, cells`
- Missing/weak reference signals: `halo-exchange, machinery, fields, pack/unpack`
- Claude Code answer excerpt: `The `HaloCells.h` header file (located at `data/raw/ippl/src/Field/HaloCells.h`) defines the `HaloCells` class, which provides functionality for **managing and exchanging halo cells** in IPPL. Key roles and features of `HaloCells` include: - **Halo Exchange Management**: It is designed to handle the "guard" or "ghos..."

file_007 - What is the purpose of FieldLayout.h?

- Verdict: **Correct**
- Correct reference answer: Defines how a global `NDIndex` domain is partitioned across MPI ranks, including local subdomains, neighbors, and communication ranges.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.92`
- Reference identifier/location hits: `1/1`
- Latency: `51.52s`
- Matched reference signals: `global, ndindex, domain, partitioned, across, mpi, ranks, local`
- Missing/weak reference signals: `neighbors`
- Claude Code answer excerpt: `The purpose of `FieldLayout.h` (located at `data/raw/ippl/src/FieldLayout/FieldLayout.h`) is to define the `FieldLayout` class template, which manages how a global `NDIndex` domain is partitioned across MPI ranks. It handles details such as: * **Domain Partitioning**: Defining how a global coordinate space is divide...

file_008 - What does ParticleBase.h define?

- Verdict: **Correct**
- Correct reference answer: Defines the generic particle container base class with positions, optional IDs, registered attributes, and a particle layout that controls migration.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.62`
- Latency: `29.70s`

- Matched reference signals: `particle, container, base, positions, ids, attributes, layout`
- Missing/weak reference signals: `generic, optional, registered, controls, migration`
- Claude Code answer excerpt: ``ParticleBase.h` defines the `ippl::ParticleBase` class (and its minimal base `ippl::ParticleBaseBase`). Specifically, it defines: - **ParticleBaseBase**: A minimal empty base class used for selection of specific variations and to support C++20 concepts like `std::derived_from`. - **ParticleBase**<PLayout, IDProper...`

file_009 - What is the purpose of ParticleSpatialLayout.h?

- Verdict: **Correct**
- Correct reference answer: Defines the spatial particle layout that decides ownership by region and migrates particles to the correct rank after motion.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.50`
- Latency: `26.71s`
- Matched reference signals: `spatial, particle, layout, region, particles, after`
- Missing/weak reference signals: `decides, ownership, migrates, correct, rank, motion`
- Claude Code answer excerpt: `The purpose of `ParticleSpatialLayout.h` (defined in `data/raw/ippl/src/Particle/ParticleSpatialLayout.h`) is to provide a specialized version of the `ParticleLayout` class that places particles on processors based on their spatial location relative to a fixed grid. Specifically: - It ensures that particles are plac...`

file_010 - What does ParticleAttrib.h implement?

- Verdict: **Correct**
- Correct reference answer: Implements the `ParticleAttrib` storage object: a Kokkos-view-backed particle attribute with resize, serialization, and mirror/access helpers.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.55`
- Reference identifier/location hits: `1/1`
- Latency: `37.16s`
- Matched reference signals: `particleattrib, storage, object, particle, attribute, serialization, ParticleAttrib`
- Missing/weak reference signals: `implements, kokkos-view-backed, resize, mirror/access, helpers`
- Claude Code answer excerpt: ``ParticleAttrib.h` defines a templated class, `ParticleAttrib`, used to represent a single particle attribute (a data element within a particle object) stored as a `Kokkos::View`. Key features and implementations include: - **Template Structure**: It is a templated class `template <typename T, class... Properties>...`

file_011 - What is UniformCartesian.h for?

- Verdict: **Correct**
- Correct reference answer: Defines the uniform Cartesian mesh class with spacing, origin, cell volume, total mesh volume, and vertex-position helpers.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.75`
- Latency: `25.53s`
- Matched reference signals: `cartesian, mesh, spacing, origin, cell, volume, total`
- Missing/weak reference signals: `uniform, vertex-position, helpers`
- Claude Code answer excerpt: ``UniformCartesian.h` defines the `UniformCartesian` class in the `ippl` namespace (located at `data/raw/ippl/src/Meshes/UniformCartesian.h`). The class is designed to represent **uniform-spacing Cartesian meshes**. It inherits from `Mesh<T, Dim>` and provides functionality for handling geometry such as: * **Mesh Spa...**

file_012 - What does OrthogonalRecursiveBisection.h do?

- Verdict: **Incorrect**
- Correct reference answer: Declares the orthogonal recursive bisection domain-decomposition/load-balancing class and its cut/median/repartition helpers.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `28.34s`
- Missing/weak reference signals: `orthogonal, recursive, bisection, domain-decomposition/load-balancing, cut/median/repartition, helpers`
- Claude Code answer excerpt: ``OrthogonalRecursiveBisection.h` (located at `data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection.h`) defines the `OrthogonalRecursiveBisection` class, which is used for domain decomposition in a parallel computing context. Key details of its functionality include: - **Purpose**: It implements a simple doma...

file_013 - What is the purpose of CIC.h in the Interpolation subsystem?

- Verdict: ****Partial****

- Correct reference answer: Declares the first-order cloud-in-cell interpolation helpers for scatter/gather between particles and grid fields.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.

- Reference-keyword coverage: `0.22`

- Latency: `32.37s`

- Matched reference signals: `interpolation, grid`

- Missing/weak reference signals: `first-order, cloud-in-cell, helpers, scatter/gather, between, particles, fields`

- Claude Code answer excerpt: `The purpose of `CIC.h` in the Interpolation subsystem is to define the core logic for **first-order/cloud-in-cell (CIC) grid interpolation**. According to the file's header comments: *** It implements first order/cloud-in-cell grid interpolation** (lines 3-4). *** The functions are currently implemented as global fu...**`

Definition Location

loc_001 - Where is FFT initialization implemented in IPPL?

- Verdict: ****Incorrect****

- Correct reference answer: FFT initialization lives in `src/FFT/FFT.hpp`, especially `FFTBase` construction/setup and the transform-specific constructors that build the heFFTe plans.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.

- Reference-keyword coverage: `0.00`

- Reference identifier/location hits: `0/3`

- Latency: `57.73s`

- Missing/weak reference signals: `fft, initialization, lives, src/fft/fft.hpp, especially, fftbase, construction/setup, transform-specific`

- Claude Code answer excerpt: `am ready to assist you with your software engineering tasks. I can help with: ****Feature Implementation:**** Adding new functionality while maintaining the existing code's style, naming conventions, and documentation standards. ****Bug Fixing:**** Identifying and resolving issues in your codebase. ****Refactoring:**** Im...`

loc_002 - Where does IPPL set up the MPI environment?

- Verdict: ****Partial****

- Correct reference answer: IPPL sets up MPI in `src/Ippl.cpp::initialize()` by constructing `mpi::Environment` and `mpi::Communicator`; the low-level `MPI\_Init` check/call is in `src/Communicate/Environment.cpp`.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.

- Reference-keyword coverage: `0.20`

- Reference identifier/location hits: `4/9`

- Latency: `43.00s`

- Matched reference signals: `mpi, mpi\_init, Environment.cpp, MPI\_Init, mpi::Environment, src/Communicate/Environment.cpp`

- Missing/weak reference signals: `sets, src/ippl.cpp::initialize, constructing, mpi::environment, mpi::communicator, low-level, check/call, src/communicate/environment.cpp`

- Claude Code answer excerpt: `In the IPPL project, the MPI environment is initialized within the `ippl::mpi::Environment` class located in `data/raw/ippl/src/Communicate/Environment.cpp`. Specifically, the constructor of this class checks if the MPI environment has already been initialized and calls `MPI_Init(&argc, &argv)` if it hasn't (lines 1...`

loc_003 - Where is Kokkos initialized and finalized inside IPPL?

- Verdict: ****Correct****

- Correct reference answer: Kokkos is initialized and finalized directly in `src/Ippl.cpp` inside `ippl::initialize()` and `ippl::finalize()`.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: `0.50`

- Reference identifier/location hits: `4/8`

- Latency: `108.96s`

- Matched reference signals: `kokkos, initialized, finalized, inside, Ippl.cpp, finalize(), initialize(), src/Ippl.cpp`

- Missing/weak reference signals: `directly, src/ippl.cpp, ippl::initialize, ippl::finalize, ippl::finalize(), ippl::initialize()`

- Claude Code answer excerpt: `IPPL, Kokkos is initialized and finalized within the `ippl` namespace in **\*\*`data/raw/ippl/src/Ippl.cpp`\*\***: **\*\*Initialization:\*\*** It is initialized using `Kokkos::initialize(argc, argv);` on line 99 inside the `initialize()` function. **\*\*Finalization:\*\*** It is finalized using `Kokkos::finalize();` on line 104 in the...

loc_004 - Where is the orthogonal recursive bisection load balancer implemented?

- Verdict: ****Incorrect****
- Correct reference answer: The ORB load balancer is implemented in ``src/Decomposition/OrthogonalRecursiveBisection.hpp`` with declarations in the matching header.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Reference identifier/location hits: ``0/2``
- Latency: ``490.72s``
- Missing/weak reference signals: ``orb, load, balancer, src/decomposition/orthogonalrecursivebisection.hpp, declarations, matching, header, OrthogonalRecursiveBisection.hpp``
- Claude Code answer excerpt: ``

loc_005 - Where are boundary conditions applied to a Field?

- Verdict: ****Incorrect****
- Correct reference answer: Field BCs are attached in ``Field::setFieldBC()`` and actually applied through ``BConds::apply()`` and the concrete face classes in ``BcTypes.hpp``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.25``
- Reference identifier/location hits: ``0/7``
- Latency: ``72.92s``
- Matched reference signals: ``field, bcs, face``
- Missing/weak reference signals: ``attached, field::setfieldbc, actually, applied, through, bconds::apply, concrete, classes``
- Claude Code answer excerpt: ``I have examined the source code in `data/raw/ippl` and identified the following components related to boundary conditions (BCs): 1. **`BConds` Class (`data/raw/ippl/src/Field/BConds.h`)**: This is a container for various BC types. It includes methods like `findBCNeighbors`, `apply`, and `assignGhostToPhysical`. 2. ***``

loc_006 - Where is the CIC (cloud-in-cell) scatter implemented?

- Verdict: ****Correct****
- Correct reference answer: CIC scatter is implemented in ``src/Interpolation/CIC.hpp``, primarily ``scatterToPoint()`` and ``scatterToField()``.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.67``
- Reference identifier/location hits: ``2/4``
- Latency: ``70.03s``
- Matched reference signals: ``cic, scatter, scatterToPoint, scatterToField, CIC.hpp, src/Interpolation/CIC.hpp``
- Missing/weak reference signals: ``src/interpolation/cic.hpp, primarily, scatterToField(), scatterToPoint()``
- Claude Code answer excerpt: ``The CIC (cloud-in-cell) scatter is implemented in the `ippl` library, specifically within the internal details of the interpolation logic. The relevant implementations are found in: **Header files**: `data/raw/ippl/src/Interpolation/CIC.h` and `data/raw/ippl/src/Interpolation/CIC.hpp`. - **Functionality**: The hea...``

loc_007 - Where is the CIC gather implemented?

- Verdict: ****Correct****
- Correct reference answer: CIC gather is implemented in ``src/Interpolation/CIC.hpp``, primarily ``gatherFromPoint()`` and ``gatherFromField()``.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.67``
- Reference identifier/location hits: ``4/4``
- Latency: ``25.73s``
- Matched reference signals: ``cic, gather, gatherFromPoint, gatherFromField, CIC.hpp, gatherFromField(), gatherFromPoint(), src/Interpolation/CIC.hpp``
- Missing/weak reference signals: ``src/interpolation/cic.hpp, primarily``
- Claude Code answer excerpt: ``CIC gather is implemented in **`data/raw/ippl/src/Interpolation/CIC.hpp`, specifically within the **`gatherFromPoint()` and **`gatherFromField()` functions.``

loc_008 - Where is halo exchange for BareField implemented?

- Verdict: ****Correct****
- Correct reference answer: Halo exchange for ``BareField`` is surfaced in ``src/Field/BareField.hpp`` and actually performed in ``src/Field/HaloCells.hpp``.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: `0.38`
- Reference identifier/location hits: `3/5`
- Latency: `73.61s`
- Matched reference signals: `halo, exchange, barefield, BareField, HaloCells.hpp, src/Field/HaloCells.hpp`
- Missing/weak reference signals: `surfaced, src/field/barefield.hpp, actually, performed, src/field/halocells.hpp, BareField.hpp, src/Field/BareField.hpp`
- Claude Code answer excerpt: `The halo exchange for `BareField` is implemented in the `HaloCells` class located in `**data/raw/ippl/src/Field/HaloCells.h**` (and its corresponding header `data/raw/ippl/src/Field/HaloCells.hpp`). Specifically: - The core logic for exchanging data of halo cells is encapsulated in the private member function `**exc...``

loc_009 - Where are the FFT-based open-boundary Poisson solver routines defined?

- Verdict: `**Incorrect**`
- Correct reference answer: The FFT open-boundary Poisson routines are defined in `src/PoissonSolvers/FFTOpenPoissonSolver.h` and `src/PoissonSolvers/FFTOpenPoissonSolver.hpp`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.17`
- Reference identifier/location hits: `0/4`
- Latency: `59.92s`
- Matched reference signals: `poisson`
- Missing/weak reference signals: `fft, open-boundary, defined, src/poissonsolvers/fftopenpoissonsolver.h, src/poissonsolvers/fftopenpoissonsolver.hpp, FFTOpenPoissonSolver.h, FFTOpenPoissonSolver.hpp, src/PoissonSolvers/FFTOpenPoissonSolver.h`
- Claude Code answer excerpt: `The `FFTOpenPoissonSolver` is an FFT-based solver for the Poisson equation with open boundaries (specifically, it calculates a scalar potential  $\phi$  from a source  $\rho$  and then finds the field  $E = -\nabla\phi$ ). Based on the header file: 1. `**Algorithms**`: It supports several algorithms: `HOCKNEY`, `VICO`, `BI...`

loc_010 - Where is the conjugate gradient solver algorithm implemented?

- Verdict: `**Partial**`
- Correct reference answer: The CG/PCG algorithm implementation is in `src/LinearSolvers/PCG.h`; the Poisson wrapper that instantiates it is `src/PoissonSolvers/PoissonCG.h`.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.25`
- Reference identifier/location hits: `2/4`
- Latency: `50.11s`
- Matched reference signals: `algorithm, implementation, PCG.h, src/LinearSolvers/PCG.h`
- Missing/weak reference signals: `cg/pcg, src/linearsolvers/pcg.h, poisson, wrapper, instantiates, src/poissonsolvers/poissoncg.h, PoissonCG.h, src/PoissonSolvers/PoissonCG.h`
- Claude Code answer excerpt: `The conjugate gradient (CG) solver algorithm is implemented in the `ippl::CG` class within the file `data/raw/ippl/src/LinearSolvers/PCG.h`. Specifically: - The base implementation for the `**conjugate gradient**` algorithm is defined in the `ippl::CG` class (starting at line 17). - A specialized version, the `**precon...``

loc_011 - Where is the FEM Poisson solver implemented?

- Verdict: `**Correct**`
- Correct reference answer: The FEM Poisson solver is implemented in `src/PoissonSolvers/FEMPoissonSolver.h`.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.75`
- Reference identifier/location hits: `2/2`
- Latency: `53.91s`
- Matched reference signals: `fem, poisson, solver, FEMPoissonSolver.h, src/PoissonSolvers/FEMPoissonSolver.h`
- Missing/weak reference signals: `src/poissonsolvers/fempoissonsolver.h`
- Claude Code answer excerpt: `The FEM Poisson solver is implemented in `data/raw/ippl/src/PoissonSolvers/FEMPoissonSolver.h`. Additionally, there are several related files in that directory: - `data/raw/ippl/src/PoissonSolvers/PreconditionedFEMPoissonSolver.h` - `data/raw/ippl/src/PoissonSolvers/PoissonCG.h` - `data/raw/ippl/src/PoissonSolvers/P...``

loc_012 - Where is the FDTD Maxwell solver base class defined?

- Verdict: `**Correct**`
- Correct reference answer: The FDTD Maxwell solver base class is defined in `src/MaxwellSolvers/FDTSolverBase.h` with implementation in `FDTSolverBase.hpp`.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.62`
- Reference identifier/location hits: `3/3`

- Latency: `29.81s`
- Matched reference signals: `fdtd, maxwell, solver, base, defined, FDTD SolverBase.h, FDTD SolverBase.hpp, src/MaxwellSolvers/FDTD SolverBase.h`
- Missing/weak reference signals: `src/maxwellsolvers/fdtdsolverbase.h, implementation, fdtdsolverbase.hpp`
- Claude Code answer excerpt: `The FDTD Maxwell solver base class is defined in`
`data/raw/ippl/src/MaxwellSolvers/FDTD SolverBase.h` (and also appears to be declared or implemented in its
corresponding header `data/raw/ippl/src/MaxwellSolvers/FDTD SolverBase.hpp`). Specifically, the file
`data/raw/ippl/src/MaxwellSolvers/FDTD SolverBase.h` contains:...

loc_013 - Where is the LagrangeSpace class defined?

- Verdict: ****Incorrect****
- Correct reference answer: The `LagrangeSpace` class is defined in `src/FEM/LagrangeSpace.h` with methods implemented in `src/FEM/LagrangeSpace.hpp`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/5`
- Latency: `101.80s`
- Missing/weak reference signals: `lagrangespace, defined, src/fem/lagrangespace.h, src/fem/lagrangespace.hpp, LagrangeSpace, LagrangeSpace.h, LagrangeSpace.hpp, src/FEM/LagrangeSpace.h`
- Claude Code answer excerpt: ``Workflow`` tool is a powerful orchestration engine designed for complex, multi-step tasks that require deterministic logic—such as loops, conditional branches, or large-scale parallel execution—that would be difficult or unreliable to manage in a single turn of conversation. When you use a workflow, you are essential...

loc_014 - Where is the particle update routine (that moves particles across ranks) implemented?

- Verdict: ****Incorrect****
- Correct reference answer: Particle migration between ranks is implemented in `ParticleSpatialLayout::update()` in `src/Particle/ParticleSpatialLayout.hpp`.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `246.52s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `particle, migration, between, ranks, particlespatiallayout::update, src/particle/particlespatiallayout.hpp`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

loc_015 - Where are IPPL timers defined?

- Verdict: ****Correct****
- Correct reference answer: IPPL timers are defined by the singleton timing layer in `src/Utility/IpplTimings.h/.cpp`, not just the low-level `Timer` helper.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.36`
- Reference identifier/location hits: `2/3`
- Latency: `21.47s`
- Matched reference signals: `timers, defined, timing, timer, IpplTimings.h, Timer`
- Missing/weak reference signals: `singleton, layer, src/utility/ippltimings.h/.cpp, not, just, low-level, helper, src/Utility/IpplTimings.h/.cpp`
- Claude Code answer excerpt: `The IPPL timers are defined in the following files: -`
`data/raw/ippl/src/Utility/Timer.h` (likely containing the base `Timer` class) -
`data/raw/ippl/src/Utility/IpplTimings.h` and `data/raw/ippl/src/Utility/IpplTimings.cpp` (likely providing the specific timer logic or management for IPPL). The presence of these fi...

Class Responsibility

cls_001 - What is the main responsibility of the BareField class?

- Verdict: ****Correct****
- Correct reference answer: `BareField` is the core distributed field storage class: it owns the data view, domain metadata, layout link, and halo operations.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.23`
- Reference identifier/location hits: `1/1`
- Latency: `45.11s`
- Matched reference signals: `barefield, core, field, BareField`
- Missing/weak reference signals: `distributed, storage, owns, view, domain, metadata, layout, link`
- Claude Code answer excerpt: `The `BareField` class is part of the IPPL codebase located in `data/raw/ippl`.

Based on my exploration of the source code: The **main responsibility of the `BareField` class** is to represent a basic, "bare" data field or property within the physical models. It acts as a container for individual quantities (like `mas...`)

cls_002 - What is the main responsibility of the Field class compared to BareField?

- Verdict: ****Correct****
- Correct reference answer: ``Field`` extends ``BareField`` with a mesh, boundary-condition container, and mesh-aware integral/average semantics.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.56``
- Reference identifier/location hits: ``2/2``
- Latency: ``57.69s``
- Matched reference signals: ``field, barefield, mesh, container, mesh-aware, BareField, Field``
- Missing/weak reference signals: ``extends, boundary-condition, integral/average, semantics``
- Claude Code answer excerpt: ``The main difference between the `Field` class and the `BareField` class lies in how they handle boundary conditions (BCs) and their relationship to a physical mesh. Based on the source code: 1. Representation of Boundary Conditions: * `Field`: Includes logic for managing boundary conditions. It maintains...`

cls_003 - What does the FieldLayout class do and what are its main template parameters?

- Verdict: ****Correct****
- Correct reference answer: ``FieldLayout<Dim>`` distributes a global ``NDIndex<Dim>`` over ranks and stores local domains, neighbors, and communication ranges; its only template parameter is ``Dim``.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.65``
- Reference identifier/location hits: ``1/3``
- Latency: ``30.60s``
- Matched reference signals: ``fieldlayout, dim, ndindex, ranks, domains, communication, ranges, template``
- Missing/weak reference signals: ``distributes, global, stores, local, neighbors, only, FieldLayout<Dim>, NDIndex<Dim>``
- Claude Code answer excerpt: ``The `FieldLayout` class in the IPPL library is responsible for describing how a given index space (represented by an `NDIndex` object) is distributed among MPI ranks. It performs the initial partitioning of these domains and handles logic related to identifying neighbor ranks, determining their spatial relationships...`

cls_004 - What is the purpose of the ParticleBase class and how is it meant to be used?

- Verdict: ****Correct****
- Correct reference answer: ``ParticleBase`` is the user-extensible base container for particles; derived classes add attributes and register them so layout-driven migration can move them.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.50``
- Reference identifier/location hits: ``1/1``
- Latency: ``27.92s``
- Matched reference signals: ``particlebase, base, container, particles, derived, classes, attributes, ParticleBase``
- Missing/weak reference signals: ``user-extensible, register, them, layout-driven, migration, move``
- Claude Code answer excerpt: ``The `ParticleBase` class serves as a base container and manager for a set of particles. Purpose: 1. Data Management: It acts as a container for particle attributes (like mass or charge). Instead of the user managing individual data fields, they define their specific properties using `ParticleAttribute<T>...`

cls_005 - What does the ParticleSpatialLayout class do?

- Verdict: ****Correct****
- Correct reference answer: ``ParticleSpatialLayout`` decides which spatial region/rank owns each particle and performs the migration/update workflow after particles move.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.46``
- Reference identifier/location hits: ``1/1``
- Latency: ``41.56s``
- Matched reference signals: ``particlespatiallayout, spatial, particle, performs, after, particles, ParticleSpatialLayout``
- Missing/weak reference signals: ``decides, region/rank, owns, each, migration/update, workflow, move``
- Claude Code answer excerpt: ``The `ParticleSpatialLayout` class (defined in `data/raw/ippl/src/Particle/ParticleSpatialLayout.hpp`) is a specialized version of `ParticleLayout` that manages the placement of particles across different processors based on their spatial locations relative to a`

fixed grid. Specifically, its responsibilities include:...

cls_006 - What is the responsibility of UniformCartesian?

- Verdict: ****Correct****
- Correct reference answer: ``UniformCartesian`` represents a uniform Cartesian mesh and exposes spacing/origin/volume and vertex-position geometry queries.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.50``
- Reference identifier/location hits: ``1/1``
- Latency: ``42.56s``
- Matched reference signals: ``uniformcartesian, represents, cartesian, mesh, geometry, UniformCartesian``
- Missing/weak reference signals: ``uniform, exposes, spacing/origin/volume, vertex-position, queries``
- Claude Code answer excerpt: ``The `UniformCartesian` class in the IPPL codebase (located in `data/raw/ippl/src/Meshes/UniformCartesian.h`) is responsible for representing **uniform-spacing Cartesian meshes**. Specifically, it provides the following functionality: * **Mesh Representation:** It inherits from `Mesh<T, Dim>` and represents a mesh wh...``

cls_007 - What does the Mesh base class define?

- Verdict: ****Correct****
- Correct reference answer: ``Mesh`` is the abstract base interface for meshes: origin, grid size, spacing, vertex positions, and volume queries.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.77``
- Reference identifier/location hits: ``1/1``
- Latency: ``597.52s``
- Matched reference signals: ``mesh, abstract, base, origin, grid, size, spacing, vertex``
- Missing/weak reference signals: ``interface, meshes, queries``
- Claude Code answer excerpt: ``The `Mesh` base class in the IPPL library serves primarily as a standard base class for all mesh types (e.g., `data/raw/ippl/src/Meshes/Mesh.h`). Its primary purpose is to allow other objects to register as users of the mesh and be notified if the mesh changes, such as when it is rescaled or restructured (`data/raw/...`)``

cls_008 - What is the responsibility of the Communicator class in ippl::mpi?

- Verdict: ****Correct****
- Correct reference answer: ``ippl::mpi::Communicator`` wraps MPI point-to-point, collectives, tag handling, and managed communication buffers.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.60``
- Reference identifier/location hits: ``0/1``
- Latency: ``84.31s``
- Matched reference signals: ``mpi, point-to-point, tag, handling, communication, buffers``
- Missing/weak reference signals: ``ippl::mpi::communicator, wraps, collectives, managed, ippl::mpi::Communicator``
- Claude Code answer excerpt: ``The `Communicator` class in `ippl::mpi` (defined in `data/raw/ippl/src/Communicate/Communicator.h`) is responsible for performing MPI communication and managing the associated infrastructure. Specifically, its responsibilities include: * **MPI Wrapper:** It provides a high-level wrapper around standard MPI functions...``

cls_009 - What is the role of the Environment class in ippl::mpi?

- Verdict: ****Partial****
- Correct reference answer: ``ippl::mpi::Environment`` manages MPI environment lifetime checks/teardown and provides an abort hook.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.38``
- Reference identifier/location hits: ``0/1``
- Latency: ``52.53s``
- Matched reference signals: ``mpi, environment, abort``
- Missing/weak reference signals: ``ippl::mpi::environment, manages, lifetime, checks/teardown, hook, ippl::mpi::Environment``
- Claude Code answer excerpt: ``The `Environment` class in the `ippl::mpi` namespace (located in `data/raw/ippl/src/Communicate/Environment.h`) serves as a manager for the MPI environment. Its primary roles include: - **Initialization and Lifecycle Management**: It provides static methods `initialized()` and `finalized()` to manage the state of th...``

cls_010 - What does the Archive class do in the Communicate module?

- Verdict: ****Correct****
- Correct reference answer: `Archive` serializes and deserializes Kokkos views and vector-valued views into raw byte buffers for MPI transfer.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.58`
- Reference identifier/location hits: `1/1`
- Latency: `42.84s`
- Matched reference signals: `archive, kokkos, views, raw, buffers, mpi, Archive`
- Missing/weak reference signals: `serializes, deserializes, vector-valued, byte, transfer`
- Claude Code answer excerpt: `The `Archive` class in the `Communicate` module is used to (de-)serialize data during MPI communication. It works with one-dimensional arrays of type `detail::Archive`, which are wrappers around Kokkos views of type `char`. These buffers contain raw bytes, which allows them to hold diverse data types. The class prov...`

cls_011 - What is the role of the BConds class?

- Verdict: ****Correct****
- Correct reference answer: `BConds` is the container of boundary-condition faces and orchestrates neighbor discovery, BC application, and ghost-to-physical assignment.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.20`
- Reference identifier/location hits: `1/1`
- Latency: `52.75s`
- Matched reference signals: `bconds, container, BConds`
- Missing/weak reference signals: `boundary-condition, faces, orchestrates, neighbor, discovery, application, ghost-to-physical, assignment`
- Claude Code answer excerpt: `The `BConds` class in the IPPL library serves as a container for boundary conditions (BCs) associated with a specific field type. According to the header file `data/raw/ippl/src/Field/BConds.h`: - It is specifically designed to hold and manage boundary condition objects (`bc_type`) of the type `detail::BCondBase<Fie...`

cls_012 - What is the role of the BaseManager class?

- Verdict: ****Correct****
- Correct reference answer: `BaseManager` provides the simulation loop skeleton: `pre\_run`, `pre\_step`, `advance`, `post\_step`, and `run(nt)`.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.89`
- Reference identifier/location hits: `5/6`
- Latency: `581.45s`
- Matched reference signals: `basemanager, simulation, loop, pre\_run, pre\_step, advance, post\_step, run`
- Missing/weak reference signals: `skeleton, run(nt)`
- Claude Code answer excerpt: `The `BaseManager` class (defined in `data/raw/ippl/src/Manager/BaseManager.h`) serves as a base class for managing simulations using the IPPL library. It provides a standardized structure and lifecycle for simulation runs, including: - **\*\*Lifecycle Hooks\*\***: It defines virtual methods to represent common stages of a s...`

cls_013 - What does the Index class represent and what operations does it support?

- Verdict: ****Correct****
- Correct reference answer: `Index` represents a regular strided integer range/slice and supports range-style arithmetic and comparison operations used in domain descriptions.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.46`
- Reference identifier/location hits: `1/1`
- Latency: `57.29s`
- Matched reference signals: `index, represents, supports, arithmetic, comparison, operations, Index`
- Missing/weak reference signals: `regular, strided, integer, range/slice, range-style, domain, descriptions`
- Claude Code answer excerpt: `The `Index` class (defined in `data/raw/ippl/src/Index/Index.h`) represents a **\*\*slice in an array\*\***, which is essentially defined as a list of **\*\*evenly spaced numbers\*\***. It can have a positive stride (increasing), a negative stride (decreasing), or a zero-stride case (which may result in a "null" interval). The clas...`

Algorithm

alg_001 - How does the FFT-based open-boundary Poisson solver compute the electrostatic potential?

- Verdict: ****Incorrect****

- Correct reference answer: The FFT open-boundary solver embeds/extends the RHS onto larger domains, FFTs both the density and a precomputed Green's function, multiplies in Fourier space, inverse transforms, and optionally computes gradients/Hessians.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.19`
- Latency: `802.70s`
- Matched reference signals: `fft, solver, ffts, green`
- Missing/weak reference signals: `open-boundary, embeds/extends, rhs, onto, larger, domains, both, density`
- Claude Code answer excerpt: `I've explored several files and performed various searches to locate the implementation of the Poisson solver in the `ippl` codebase. Based on my investigation: 1. **FieldSolver.hpp**: This file (at `data/raw/ippl/alpine/FieldSolver.hpp`) serves as a high-level wrapper for different types of solvers, including "FFT"...

alg_002 - How does the FFT-based periodic Poisson solver work in IPPL?

- Verdict: **Partial**
- Correct reference answer: The periodic FFT solver FFTs the charge density, divides each Fourier mode by $|k|^2$ (with the zero mode handled separately), then inverse transforms; gradients are computed by multiplying by $ik/|k|^2$ before the inverse FFT.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.50`
- Reference identifier/location hits: `0/2`
- Latency: `462.82s`
- Matched reference signals: `periodic, fft, solver, each, fourier, mode, inverse, multiplying`
- Missing/weak reference signals: `ffts, charge, density, divides, zero, handled, separately, then`
- Claude Code answer excerpt: `The FFT-based periodic Poisson solver in IPPL works by utilizing the Fourier transform to solve the Poisson equation on a periodic domain. The implementation details are primarily found in `data/raw/ippl/src/PoissonSolvers/FFTPeriodicPoissonSolver.h`. **Core Mechanism:** 1. **Forward Transform**: The input field (th...

alg_003 - How does the truncated Green periodic Poisson solver differ from the standard periodic c

- Verdict: **Partial**
- Correct reference answer: The truncated-Green periodic solver replaces the simple $1/|k|^2$ kernel with the FFT of a truncated/erf-smoothed Green's function $\text{forceConstant} * \text{erf}(\alpha r) / r$, then follows the same transform-multiply-inverse pattern.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.29`
- Reference identifier/location hits: `0/3`
- Latency: `280.67s`
- Matched reference signals: `periodic, green, forceconstant, erf, alpha`
- Missing/weak reference signals: `truncated-green, solver, replaces, simple, kernel, fft, truncated/erf-smoothed, then`
- Claude Code answer excerpt: `I have reviewed the implementation of `FFTTruncatedGreenPeriodicPoissonSolver` and `FFTPeriodicPoissonSolver`. The `FFTTruncatedGreenPeriodicPoissonSolver` implements a Green's function-based approach for periodic boundaries, which is typically used to handle short-range interactions in schemes like Particle-Particl...

alg_004 - How does IPPL's CG solver iterate in the PCG.h implementation?

- Verdict: **Correct**
- Correct reference answer: In `PCG.h`, the CG iteration forms residual/direction fields, applies the operator, computes `alpha` and `beta` from inner products, updates the solution/residual/direction, and stops on tolerance or iteration count; the preconditioned variant inserts M^{-1} applications on the residual.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.48`
- Reference identifier/location hits: `3/4`
- Latency: `50.40s`
- Matched reference signals: `pcg.h, iteration, fields, operator, alpha, beta, inner, tolerance`
- Missing/weak reference signals: `forms, residual/direction, applies, computes, products, updates, solution/residual/direction, stops`
- Claude Code answer excerpt: `The IPPL's Conjugate Gradient (CG) solver, as implemented in `PCG.h`, follows a standard Preconditioned Conjugate Gradient algorithm. In the context of the `PCG` class (which inherits from `CG`), it incorporates a preconditioner to improve convergence. **Implementation Overview** The core logic is found in the `oper...

alg_005 - How is cloud-in-cell interpolation performed in IPPL (both scatter and gather directions

- Verdict: **Incorrect**

- Correct reference answer: CIC computes lower/upper weights per axis, scatters with atomic adds to the `2^Dim` neighboring grid points, and gathers by summing the same weighted stencil back to particles.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.26`
- Reference identifier/location hits: `0/1`
- Latency: `122.72s`
- Matched reference signals: `cic, weights, grid, same, weighted`
- Missing/weak reference signals: `computes, lower/upper, per, axis, scatters, atomic, dim, neighboring`
- Claude Code answer excerpt: `The "CIC" (Cloud-In-Cell) interpolation in IPPL is handled through the interaction between ParticleContainer and FieldContainer (or more specifically, their underlying buffer management). In both scatter and gather directions: 1. Gathering (Interpolating a field at a particle position): When a particle's loc...`

alg_006 - How does the standard FDTD solver advance the electromagnetic field in time?

- Verdict: **Correct**
- Correct reference answer: The standard FDTD solver advances the source-like field with a second-order finite-difference update using the previous and current time levels plus neighboring cells and source terms, then fills halos and reapplies BCs.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.55`
- Latency: `103.31s`
- Matched reference signals: `standard, ftdt, solver, advances, field, update, previous, current`
- Missing/weak reference signals: `source-like, second-order, finite-difference, levels, neighboring, then, fills, halos`
- Claude Code answer excerpt: `The standard FDTD solver advances the electromagnetic field in time by performing a single time step update using a specific weighted sum of the current, previous, and source fields. Specifically, as implemented in StandardFDTDSolver::step() (lines 33-86 of data/raw/ippl/src/MaxwellSolvers/StandardFDTDSolver.hpp...`

alg_007 - How does the LagrangeSpace class evaluate degrees of freedom per element?

- Verdict: **Incorrect**
- Correct reference answer: `LagrangeSpace` evaluates element DOFs by mapping local element indices to global DOF indices, then evaluates order-1 reference-element shape functions and gradients on those local DOFs.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/1`
- Latency: `70.15s`
- Missing/weak reference signals: `lagrangespace, evaluates, element, dofs, mapping, local, indices, global`
- Claude Code answer excerpt: `I have received the request and am ready to assist. I see a large codebase related to "ippl" and various scripts for model evaluation and RAG (Retrieval-Augmented Generation). Please let me know what specific task you would like me to perform or which part of the system you would like to investigate first.`

alg_008 - How is Gauss-Jacobi quadrature implemented and used in IPPL FEM?

- Verdict: **Correct**
- Correct reference answer: Gauss-Jacobi quadrature computes nodes/weights by Newton iteration on Jacobi polynomial roots, starting from Chebyshev or LehrFEM-style guesses, and uses the resulting rule on reference elements in FEM.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.67`
- Latency: `32.45s`
- Matched reference signals: `gauss-jacobi, quadrature, newton, iteration, jacobi, polynomial, roots, starting`
- Missing/weak reference signals: `computes, nodes/weights, lehrfem-style, resulting, reference, elements`
- Claude Code answer excerpt: `Gauss-Jacobi quadrature is implemented in IPPL FEM via the GaussJacobiQuadrature class template (found in data/raw/ippl/src/FEM/Quadrature/GaussJacobiQuadrature.h and .hpp). Implementation Details: 1. Base Class: The implementation follows a base class Quadrature<T, NumNodes1D, ElementType> and provi...`

alg_009 - How does the orthogonal recursive bisection decide where to cut the domain?

- Verdict: **Correct**
- Correct reference answer: ORB chooses a cut axis from the domain geometry and then finds a weight-balanced cut/median so recursive splits distribute load more evenly.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.44`

- Latency: `188.08s`
- Matched reference signals: `orb, cut, axis, domain, then, finds, recursive`
- Missing/weak reference signals: `chooses, geometry, weight-balanced, cut/median, splits, distribute, load, more`
- Claude Code answer excerpt: `The orthogonal recursive bisection (ORB) algorithm decides where to cut the domain based on a multi-step process focused on ensuring balanced weights and selecting the most appropriate axis: 1. **Find Cut Axis**: The algorithm first identifies which dimension to split by finding the **longest axis** of the current d...`

alg_010 - How does inverse transform sampling generate particles from a given distribution in IPPL

- Verdict: **Correct**
- Correct reference answer: Inverse transform sampling maps uniform random values in CDF space to samples by estimating the inverse and refining it with Newton-Raphson, dimension by dimension.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.75`
- Latency: `29.15s`
- Matched reference signals: `inverse, transform, sampling, uniform, random, values, cdf, space`
- Missing/weak reference signals: `maps, samples, estimating, refining`
- Claude Code answer excerpt: `The inverse transform sampling process in IPPL for generating particles from a given distribution is implemented in the `ippl::random::InverseTransformSampling` class (located in `data/raw/ippl/src/Random/InverseTransformSampling.h`). The core logic follows these steps: 1. **Bound Initialization and Normalization**...

alg_011 - How does the FFT class interface with heFFTe to perform complex-to-complex and real-to-complex transforms?

- Verdict: **Partial**
- Correct reference answer: The FFT layer builds heFFTe boxes/plans in `setup()`, copies IPPL field data into the required views/subviews, calls heFFTe forward/backward transforms, and copies results back for CC/RC/DST/DCT variants.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.42`
- Reference identifier/location hits: `0/1`
- Latency: `32.72s`
- Matched reference signals: `fft, heffte, setup, field, calls, transforms, back`
- Missing/weak reference signals: `layer, builds, boxes/plans, copies, required, views/subviews, forward/backward, results`
- Claude Code answer excerpt: `The `FFT` class serves as an interface between the IPPL library and the `heffte` library for performing Fourier transforms. It handles both **complex-to-complex** (using the `CCTransform` template parameter) and **real-to-complex** (using the `RCTransform` template parameter) transformations. **Core Implementation**...

alg_012 - How does IPPL's preconditioned CG use the preconditioner defined in Preconditioner.h?

- Verdict: **Partial**
- Correct reference answer: PCG applies a chosen preconditioner from `Preconditioner.h` to the residual, uses the preconditioned residual in its inner products and direction updates, and iterates until tolerance/max iterations.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.06`
- Reference identifier/location hits: `1/1`
- Latency: `27.19s`
- Matched reference signals: `preconditioner, Preconditioner.h`
- Missing/weak reference signals: `pcg, applies, chosen, preconditioner.h, residual, preconditioned, inner, products`
- Claude Code answer excerpt: `I have reviewed `data/raw/ippl/src/LinearSolvers/Preconditioner.h`. It defines various preconditioners such as Jacobi, Polynomial Newton, Polynomial Chebyshev, and Richardson (including an alternative version) and a 2-step Gauss-Seidel preconditioner. Based on the initial files provided, I see that: 1. The library i...

Data Flow

flow_001 - How does charge density flow from particles to the grid in a typical PIC step in IPPL?

- Verdict: **Incorrect**
- Correct reference answer: In the PIC workflow, particle charges and positions are deposited onto the grid with CIC (`scatter`) or FEM assembly (`assemble_rhs_from_particles`), accumulated into `rho`, and then normalized/background-shifted in the manager before solving.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.

- Reference-keyword coverage: `0.00`
- Latency: `350.49s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `pic, workflow, particle, charges, positions, deposited, onto, grid`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

flow_002 - How does the electric field flow from the grid back to particles after a Poisson solve?

- Verdict: ****Partial****
- Correct reference answer: After the field solve, the manager gathers the electric field back to particles with CIC `gather(...)` or FEM `interpolate\_grad\_to\_diracs(...)` , storing particle-local `E` values for the push step.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.39`
- Reference identifier/location hits: `0/2`
- Latency: `65.19s`
- Matched reference signals: `after, field, solve, electric, particles, fem`
- Missing/weak reference signals: `manager, gathers, back, cic, gather, interpolate\_grad\_to\_diracs, storing, particle-local`
- Claude Code answer excerpt: `Based on the implementation in `data/raw/ippl/alpine/FieldSolver.hpp`, the way electric fields are handled after a Poisson solve depends on the specific solver type being used: 1. **\*\*For CG, PCG, FEM, and FEM\_PRECON solvers\*\***: These solvers are configured to compute the **\*\*potential ( $\phi$ )\*\*** directly. The electric f...`

flow_003 - How are halo cells exchanged between neighboring MPI ranks for a BareField?

- Verdict: ****Partial****
- Correct reference answer: Halo exchange uses `FieldLayout` neighbor/range metadata to pack field subviews, send/receive them with MPI, unpack into ghost or physical regions, and handle serial-periodic dimensions locally.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.32`
- Reference identifier/location hits: `0/1`
- Latency: `31.53s`
- Matched reference signals: `halo, exchange, pack, mpi, unpack, dimensions`
- Missing/weak reference signals: `fieldlayout, neighbor/range, metadata, field, subviews, send/receive, them, ghost`
- Claude Code answer excerpt: `The halo cells of a `BareField` are exchanged between neighboring MPI ranks using the `HaloCells` class, which implements the logic in `data/raw/ippl/src/Field/HaloCells.hpp`. **### Mechanism Summary: 1. **\*\*Exchanges\*\*****: The primary method for exchange is `exchangeBoundaries`, which is called by higher-level functions l...`

flow_004 - How are particles migrated between MPI ranks by ParticleSpatialLayout?

- Verdict: ****Incorrect****
- Correct reference answer: `ParticleSpatialLayout::update()` applies particle BCs, locates destination ranks, advertises receive counts with RMA, sends all registered attributes, destroys invalid local particles, and receives the incoming ones.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.09`
- Reference identifier/location hits: `0/3`
- Latency: `57.02s`
- Matched reference signals: `local, particles`
- Missing/weak reference signals: `particlespatiallayout::update, applies, particle, bcs, locates, destination, ranks, advertises`
- Claude Code answer excerpt: `I have analyzed the codebase and documentation related to `ParticleSpatialLayout`. The primary source for this information is **\*\*`data/raw/ippl/ParticleSpatialLayout.h`\*\***. Based on my search across various evaluative files (like `experiments/manual_questions.json` and `gwendolen_cc_eval.out`), here are the key findin...`

flow_005 - How does FieldLayout decide which index ranges live on which rank?

- Verdict: ****Partial****
- Correct reference answer: `FieldLayout` uses the partitioner and the requested parallel/serial decomposition flags to assign each rank a local `NDIndex`, plus neighbor and communication range tables.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.20`
- Reference identifier/location hits: `1/2`

- Latency: `61.82s`
- Matched reference signals: `fieldlayout, neighbor, communication, FieldLayout`
- Missing/weak reference signals: `partitioner, requested, parallel/serial, decomposition, flags, assign, each, rank`
- Claude Code answer excerpt: `I've explored the codebase for information on how `FieldLayout` and its internal components are structured. I see that `FieldLayout` handles domain partitioning, neighbor logic, and communication mapping for MPI systems, while `SubFieldLayout` extends this to handle sub-regions of a larger domain. To better understa...`

flow_006 - How is a ParameterList used to configure the FFT and solver classes?

- Verdict: ****Incorrect****
- Correct reference answer: `ParameterList` is merged into FFT and solver objects, then consumed by `setup()/initialize()/solve()` for communication mode, pencil/reorder flags, transform direction, solver type, tolerances, preconditioner settings, and output type.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `104.65s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `parameterlist, merged, fft, solver, objects, then, consumed, setup`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

flow_007 - How does data flow through the BaseManager's pre_step / advance / post_step hooks during simulation?

- Verdict: ****Correct****
- Correct reference answer: `BaseManager::run()` executes `pre\_step()`, `advance()`, and `post\_step()` for each timestep; derived managers use those hooks for deposit/solve/gather, time updates, and output.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.56`
- Reference identifier/location hits: `4/6`
- Latency: `24.74s`
- Matched reference signals: `basemanager::run, pre\_step, advance, post\_step, each, derived, hooks, time`
- Missing/weak reference signals: `executes, timestep, managers, use, those, deposit/solve/gather, output, BaseManager::run()`
- Claude Code answer excerpt: `The `BaseManager` class in the IPPL library (defined in `data/raw/ippl/src/Manager/BaseManager.h`) provides a standard structure for executing simulations through a sequence of method calls within a loop. Data flows through the following hooks as follows: 1. **\*\*pre\_step()\*\***: This is called before each step of the s...`

flow_008 - How does the FEM solver move data between fields and FEMVector representations?

- Verdict: ****Incorrect****
- Correct reference answer: FEM solvers move data by assembling field data into FEM load/stiffness representations (`evaluateLoadVector`, `evaluateAx`, `FEMVector`) and then interpolating the solved field quantities back where needed.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `237.71s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `fem, solvers, move, assembling, field, fem, load/stiffness, representations`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

flow_009 - How does OrthogonalRecursiveBisection redistribute particles after rebalancing?

- Verdict: ****Partial****
- Correct reference answer: After ORB recomputes the decomposition, particle ownership changes are realized by updating the layout/regions and then migrating particles with the usual particle update path.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.24`
- Latency: `41.49s`
- Matched reference signals: `particle, then, particles`
- Missing/weak reference signals: `after, orb, recomputes, decomposition, ownership, changes, realized, updating`
- Claude Code answer excerpt: `The `OrthogonalRecursiveBisection` algorithm (located in `data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection.hpp`) manages the redistribution of particles during the rebalancing process by utilizing a weight-based accumulation and partitioning scheme. Specifically, the distribution follows these steps: 1....`

Api Usage

api_001 - How do I create a 3D Field of doubles in IPPL?

- Verdict: ****Correct****

- Correct reference answer: A typical 3D field is ``Field<double, 3, UniformCartesian<double, 3>, Cell>`` constructed from a ``UniformCartesian`` mesh and a ``FieldLayout<3>``.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: ``0.64``

- Reference identifier/location hits: ``1/3``

- Latency: ``152.86s``

- Matched reference signals: ``field, double, uniformcartesian, mesh, UniformCartesian``

- Missing/weak reference signals: ``typical, cell, constructed, fieldlayout, Field<double, 3, UniformCartesian<double, 3>, Cell>, FieldLayout<3>``

- Claude Code answer excerpt: ``To create a 3D field of doubles in IPPL, you typically have two options depending on whether you need specialized features like mesh-aware properties and boundary conditions. ### Option 1: Using `BareField` (Basic Data Structure) If you only need a raw multidimensional array with ghost cells but don't require automa...``

api_002 - How do I construct a UniformCartesian mesh with custom spacing and origin?

- Verdict: ****Correct****

- Correct reference answer: Construct the mesh as ``UniformCartesian<double, Dim> mesh(domain, hx, origin);`` where ``hx`` is the spacing vector and ``origin`` is the mesh origin.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: ``0.85``

- Reference identifier/location hits: ``2/4``

- Latency: ``25.14s``

- Matched reference signals: ``construct, mesh, uniformcartesian, dim, origin, spacing, vector, hx``

- Missing/weak reference signals: ``double, domain, UniformCartesian<double, Dim> mesh(domain, hx, origin), mesh(domain, hx, origin)``

- Claude Code answer excerpt: ``To construct a `UniformCartesian` mesh with custom spacing and origin, you can use either the constructor that accepts these parameters directly or the `initialize` method. In the `ippl::UniformCartesian<T, Dim>` class (defined in `data/raw/ippl/src/Meshes/UniformCartesian.h` and `data/raw/ippl/src/Meshes/UniformCar...``

api_003 - How do I attach a boundary condition (e.g. periodic) to a Field?

- Verdict: ****Incorrect****

- Correct reference answer: Create a ``BConds<FieldT, Dim>``, assign per-face BC objects such as ``PeriodicFace<FieldT>``, then call ``field.setFieldBC(bc)``.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.

- Reference-keyword coverage: ``0.23``

- Reference identifier/location hits: ``0/4``

- Latency: ``30.67s``

- Matched reference signals: ``bconds, objects, periodicface``

- Missing/weak reference signals: ``create, fieldt, dim, assign, per-face, such, then, call``

- Claude Code answer excerpt: ``To attach a boundary condition like periodic to a `Field`, you should use the `PeriodicFace` class defined in `src/Field/BcTypes.h`. ### Implementation Details: - **Class**: `ippl::PeriodicFace<Field>` (Line 158, `src/Field/BcTypes.h`) - **Constructor**: The constructor takes an unsigned integer representing...``

api_004 - How do I define a custom particle class that adds mass and velocity attributes on top of ParticleBase?

- Verdict: ****Correct****

- Correct reference answer: Define a class derived from ``ParticleBase<ParticleSpatialLayout<...>>``, add ``mass`` and ``velocity`` attributes as members, and register them with ``addAttribute()`` in the constructor.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: ``0.83``

- Reference identifier/location hits: ``2/4``

- Latency: ``27.47s``

- Matched reference signals: ``define, derived, particlebase, particlespatiallayout, mass, velocity, attributes, members``

- Missing/weak reference signals: ``register, them, ParticleBase<ParticleSpatialLayout<...>>, addAttribute()``

- Claude Code answer excerpt: ``To define a custom particle class with additional attributes (like mass and velocity) that inherits from `ParticleBase`, you should follow the pattern shown in the `ParticleBase.hpp` header file (lines 31-50). Specifically, you should: 1. Define a new class derived from `ParticleBase`. The template arguments for `Pa...``

api_005 - How do I call `ippl::initialize` and `ippl::finalize` in a main program?

- Verdict: ****Correct****
- Correct reference answer: Call the free namespace functions ``ippl::initialize(argc, argv);`` and ``ippl::finalize();`` from ``main``, as shown in the docs/examples.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.40``
- Reference identifier/location hits: ``5/7``
- Latency: ``14.87s``
- Matched reference signals: ``call, ippl::initialize, ippl::finalize, main, finalize(), ippl::finalize()``
- Missing/weak reference signals: ``free, namespace, argc, argv, shown, docs/examples, initialize(argc, argv), ippl::initialize(argc, argv)``
- Claude Code answer excerpt: ``To call `ippl::initialize` and `ippl::finalize` in a main program, you should follow the pattern typically used in initialization routines of the library. Based on the source code (e.g., see `data/raw/ippl/doc/examples/HelloWorld.hpp`), these functions are called at the beginning and end of the application's executi...``

api_006 - How do I set up and run a forward FFT on an IPPL Field?

- Verdict: ****Partial****
- Correct reference answer: Set up the relevant layouts/fields/parameters, build an ``FFT<CCTransform, ...>`` or ``FFT<RCTransform, ...>`` object, and call ``transform(FORWARD, ...)`` on the field(s).
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.31``
- Reference identifier/location hits: ``0/4``
- Latency: ``83.11s``
- Matched reference signals: ``fft, object, field``
- Missing/weak reference signals: ``set, relevant, layouts/fields/parameters, build, cctransform, rctransform, call, transform``
- Claude Code answer excerpt: ``The `BareField` class is a fundamental building block in the `ippl` library, representing a raw data field with specific dimensions and memory layout. It includes functionality for: - **Data Access & Manipulation**: Support for standard operations (sum, max, min, prod), view management, and range policies. - **Memor...``

api_007 - How do I construct an `FFTPeriodicPoissonSolver` and solve for the potential?

- Verdict: ****Partial****
- Correct reference answer: Construct ``FFTPeriodicPoissonSolver(lhs, rhs)`` (or default construct then set fields/params), ensure the RHS is attached, and call ``solve()`` to write the potential/gradient outputs.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.18``
- Reference identifier/location hits: ``1/2``
- Latency: ``55.28s``
- Matched reference signals: ``fftperiodicpoissonsolver, call, solve, solve()``
- Missing/weak reference signals: ``construct, lhs, rhs, default, then, set, fields/params, ensure``
- Claude Code answer excerpt: ``The `FFTPeriodicPoissonSolver` class can be constructed and used to solve for the potential as follows, based on the implementation in `data/raw/ippl/alpine/FieldSolver.hpp`: 1. **Constructor**: Initialize the solver with a `FieldContainer` (representing the source distribution) and dimensions. ``cpp auto solver =...``

api_008 - How do I invoke the conjugate gradient Poisson solver on a Field?

- Verdict: ****Correct****
- Correct reference answer: Use ``PoissonCG<FieldT>`` (or the Alpine ``FieldSolver`` wrapper around it), set its fields/parameters, and call ``solve()`` so the internal CG/PCG object iterates on the field.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.31``
- Reference identifier/location hits: ``2/3``
- Latency: ``65.67s``
- Matched reference signals: ``use, fieldsolver, solve, internal, field, FieldSolver, solve()``
- Missing/weak reference signals: ``poissoncg, fieldt, alpine, wrapper, around, set, fields/parameters, call``
- Claude Code answer excerpt: ``To invoke the conjugate gradient Poisson solver on a `Field`, you should use the **`CGSolver_t`** (or its preconditioned variant, **`PCGSolver_t`**) via the `FieldSolver` class. In the IPPL framework, the "conjugate gradient" method for solving the Poisson equation is implemented in the `FieldSolver` class (found in...``

api_009 - How do I sample particles from a normal distribution using IPPL Random utilities?

- Verdict: ****Partial****
- Correct reference answer: For Gaussian draws, IPPL exposes both ``randn`` convenience generation and the more general ``InverseTransformSampling`` + ``NormalDistribution`` path.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.08``
- Reference identifier/location hits: ``1/3``
- Latency: ``51.34s``
- Matched reference signals: ``normaldistribution, NormalDistribution``
- Missing/weak reference signals: ``gaussian, draws, exposes, both, randn, convenience, generation, more``
- Claude Code answer excerpt: ``To sample from a normal distribution in IPPL, you should use the `NormalDistribution` class located in `data/raw/ippl/src/Random/NormalDistribution.h`. ### Implementation Details: - **Class**: `ippl::random::NormalDistribution<T, Dim>` - **Constructor**: Takes a pointer to an array of parameters (`const T* par_p`)....``

api_010 - How do I register particle attributes with ParticleBase so that they are communicated on migration?

- Verdict: ****Incorrect****
- Correct reference answer: Register each particle attribute with ``addAttribute(...)``; migration then serializes and transfers all registered attributes automatically.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.08``
- Reference identifier/location hits: ``0/1``
- Latency: ``46.11s``
- Matched reference signals: ``particle``
- Missing/weak reference signals: ``register, each, attribute, addattribute, migration, then, serializes, transfers``
- Claude Code answer excerpt: ``logic for creating a particle with a specific ID is implemented in the `createWithID` method within the `ParticleBase` class. This can be found in the header file: **File:** `data/raw/ippl/src/Particle/ParticleBase.hpp` **Lines:** 120–135 ``cpp 120 template <class PLayout, typename... IP> 121 void ParticleBase<PLayout...``

api_011 - How do I access the MPI rank and size through the IPPL Communicator?

- Verdict: ****Correct****
- Correct reference answer: Access rank and size with ``ippl::Comm->rank()`` and ``ippl::Comm->size()`` (or the same methods on a ``Communicator`` object).
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.60``
- Reference identifier/location hits: ``3/6``
- Latency: ``23.82s``
- Matched reference signals: ``access, rank, size, communicator, Communicator, rank(), size()``
- Missing/weak reference signals: ``ippl::comm-, same, object, ippl::Comm, ippl::Comm->rank(), ippl::Comm->size()``
- Claude Code answer excerpt: ``The IPPL `Communicator` class provides member functions to access the MPI rank and size: - **Rank:** Use the `.rank()` method. - **Size:** Use the `.size()` method. These are defined in `data/raw/ippl/src/Communicate/Communicator.h`: - Line 45: `int size() const noexcept { return size_m; }` - Line 47: `int rank() co...``

api_012 - How do I use IpplTimings to measure a region of code?

- Verdict: ****Correct****
- Correct reference answer: Get a timer handle with ``IpplTimings::getTimer("name")``, surround the code with ``startTimer()`` / ``stopTimer()``, and print with ``IpplTimings::print()``.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.50``
- Reference identifier/location hits: ``4/8``
- Latency: ``18.16s``
- Matched reference signals: ``timer, ippltimings::gettimer, name, print, ippltimings::print, IpplTimings::getTimer, IpplTimings::print, IpplTimings::print()``
- Missing/weak reference signals: ``get, handle, surround, starttimer, stoptimer, IpplTimings::getTimer("name"), getTimer("name"), startTimer()``
- Claude Code answer excerpt: ``To use `IpplTimings` to measure a region of code, follow these steps as outlined in the source code and comments: 1. **Create/Get a Timer**: Obtain a reference to a timer using a descriptive name. This will either create a new timer or return a reference to an existing one with that name. ``cpp IpplTimings::TimerRe...``

Parallelism And Kokkos

par_001 - How does IPPL achieve performance portability across CPUs and GPUs?

- Verdict: ****Correct****

- Correct reference answer: IPPL gets performance portability by combining Kokkos execution/memory abstractions with MPI and heFFTe, so the same code paths can target CPUs and GPUs.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: `0.57`

- Latency: `45.22s`

- Matched reference signals: `performance, portability, kokkos, mpi, heffte, same, cpus, gpus`

- Missing/weak reference signals: `gets, combining, execution/memory, abstractions, paths, target`

- Claude Code answer excerpt: `PL achieves performance portability across CPUs and GPUs primarily by leveraging the ****Kokkos**** programming model, alongside ****MPI**** for distributed memory parallelism and ****HeFFTe**** for high-performance FFTs. Specifically, the implementation details include: 1. ****Abstraction via Kokkos:**** IPPL utilizes Kokkos to ab...`

par_002 - Why does IPPL use Kokkos::View for field storage?

- Verdict: ****Correct****

- Correct reference answer: IPPL uses `Kokkos::View` because fields need execution-space-aware storage, subviews, mirrors, and portable kernels across host/device memory spaces.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: `0.50`

- Reference identifier/location hits: `1/1`

- Latency: `53.04s`

- Matched reference signals: `kokkos::view, because, fields, storage, kernels, memory, spaces, Kokkos::View`

- Missing/weak reference signals: `need, execution-space-aware, subviews, mirrors, portable, across, host/device`

- Claude Code answer excerpt: `at-specific capabilities`, such as lookups and neighbor searches, because it allows algorithms to be written using standard loops or kernels that are then automatically optimized for different hardware. Specifically: * ****Particle Attributes****: In `data/raw/ippl/src/Particle/ParticleAttrib.hpp`, data is stored in a...`

par_003 - How does IPPL parallelize loops over field indices using Kokkos RangePolicy?

- Verdict: ****Correct****

- Correct reference answer: Loops are parallelized through `ippl::getRangePolicy` / `createRangePolicy` and the `ippl::parallel\_for` wrapper over Kokkos `RangePolicy`/`MDRangePolicy`.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: `0.70`

- Reference identifier/location hits: `5/5`

- Latency: `179.77s`

- Matched reference signals: `loops, through, ippl::getrangepolicy, ippl::parallel\_for, wrapper, kokkos, rangepolicy, MDRangePolicy`

- Missing/weak reference signals: `parallelized, createrangepolicy, mdrangepolicy`

- Claude Code answer excerpt: `To answer the question about how IPPL parallelizes loops over field indices using Kokkos `RangePolicy`, I have examined `data/raw/ippl/src/Utility/ParallelDispatch.h`. IPPL uses a wrapper structure called `RangePolicy` to encapsulate and simplify Kokkos range policies: 1. **\*\*Wrapper Structure\*\***: The `ippl::RangePolic...`

par_004 - How does IPPL support mixed precision (float vs double) for fields?

- Verdict: ****Incorrect****

- Correct reference answer: Precision is chosen by the field value type (`Field<float,...>` vs `Field<double,...>`), and the codebase also contains mixed-precision examples where fields/particles deliberately use different precisions.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.

- Reference-keyword coverage: `0.00`

- Reference identifier/location hits: `0/2`

- Latency: `99.65s`

- Missing/weak reference signals: `precision, chosen, field, value, type, float, double, codebase`

- Claude Code answer excerpt: ``

par_005 - How does IPPL handle multiple execution spaces like OpenMP and CUDA in the same build?

- Verdict: ****Incorrect****

- Correct reference answer: Multiple execution spaces are enabled at build time through `IPPL\_PLATFORMS` and Kokkos/HeFFTe backend flags; IPPL also has container helpers for multi-space data.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.

- Reference-keyword coverage: `0.20`
- Reference identifier/location hits: `0/1`
- Latency: `214.65s`
- Matched reference signals: `multiple, execution, spaces`
- Missing/weak reference signals: `enabled, build, time, through, ippl\_platforms, kokkos/heffte, backend, flags`
- Claude Code answer excerpt: `PL handles multiple execution spaces (such as OpenMP and CUDA) by leveraging **Kokkos** as its primary abstraction layer for performance portability. Specifically: - **Abstraction via Kokkos**: IPPL uses Kokkos to provide a unified programming model that can target different architectures (CPUs, GPUs) and execution...`

par_006 - What is the purpose of ParallelDispatch.h and what helpers does it expose?

- Verdict: **Correct**
- Correct reference answer: `ParallelDispatch.h` wraps Kokkos policies and functors, exposing `RangePolicy`, `getRangePolicy`, `createRangePolicy`, `parallel\_for`, and `parallel\_reduce`.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.91`
- Reference identifier/location hits: `6/6`
- Latency: `20.30s`
- Matched reference signals: `paralleldispatch.h, wraps, kokkos, policies, functors, rangepolicy, getrangepolicy, createrangepolicy`
- Missing/weak reference signals: `exposing`
- Claude Code answer excerpt: `The header file `ParallelDispatch.h` (located at `data/raw/ippl/src/Utility/ParallelDispatch.h`) provides utility functions related to parallel dispatch in the IPPL library, specifically wrapping and simplifying interactions with Kokkos' parallel execution models. It exposes several key helpers and structures: **C...**`

par_007 - How does the Archive class serialize/deserialize Kokkos views for MPI transfer?

- Verdict: **Partial**
- Correct reference answer: `Archive` serializes Kokkos views into one-dimensional byte buffers with Kokkos kernels and `memcpy`, then deserializes them back into views on receive.
- Explanation: The answer contains some relevant information but also indicates uncertainty or inability to fully answer.
- Reference-keyword coverage: `0.50`
- Reference identifier/location hits: `2/2`
- Latency: `25.44s`
- Matched reference signals: `archive, kokkos, views, one-dimensional, byte, buffers, Archive, memcpy`
- Missing/weak reference signals: `serializes, kernels, memcpy, then, deserializes, them, back, receive`
- Claude Code answer excerpt: `The `Archive` class, located in `data/raw/ippl/src/Communicate/Archive.h` and implemented in `data/raw/ippl/src/Communicate/Archive.hpp`, manages the serialization and deserialization of data (including Kokkos views) for MPI communication. The core mechanism for handling Kokkos views involves: 1. **Buffer Type**: It...`

par_008 - How are MPI tags managed in IPPL to avoid collisions across subsystems?

- Verdict: **Correct**
- Correct reference answer: MPI tags are managed with fixed base tags in `Tags.h` and the cyclic tag allocator in `TagMaker`, which `Communicator` inherits.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.62`
- Reference identifier/location hits: `2/3`
- Latency: `36.54s`
- Matched reference signals: `mpi, tags, managed, base, tags.h, tag, tagmaker, TagMaker`
- Missing/weak reference signals: `fixed, cyclic, allocator, communicator, inherits, Communicator`
- Claude Code answer excerpt: `The `TagMaker.h` file describes a mechanism for managing MPI tags to avoid collisions between different subsystems. 1. **Base Tags**: Each subsystem or functionality (e.g., "Halo", "Particle") is assigned a unique **base tag** from the `Tag` enum defined in `Tags.h`. 2. **Cycle Logic**: The `TagMaker` class maintain...`

par_009 - What does Kokkos::fence do, and when does IPPL call the wrapper ippl::fence?

- Verdict: **Partial**
- Correct reference answer: `Kokkos::fence()` waits for outstanding kernels; IPPL exposes `ippl::fence()` as a runtime wrapper in `Ippl.cpp` and also fences internally after many kernel/data-movement steps.
- Explanation: The answer contains some relevant information but also indicates uncertainty or inability to fully answer.
- Reference-keyword coverage: `0.13`
- Reference identifier/location hits: `1/6`
- Latency: `259.99s`
- Matched reference signals: `ippl::fence, fences`

- Missing/weak reference signals: `kokkos::fence, waits, outstanding, kernels, exposes, runtime, wrapper, ippl.cpp`
- Claude Code answer excerpt: `I've tried several methods to find `ippl::fence` and related terms in `data/raw/ippl`, but none of the standard search tools (Grep, Glob) returned any results. This suggests that either "fence" is not used as a literal string in those files or it may be inside a header file that doesn't have a ".h" extension, or it...`

Boundary And Halo

bc_001 - What boundary condition types are provided in BcTypes.h?

- Verdict: ****Correct****
- Correct reference answer: `BcTypes.h` provides periodic, constant, zero, extrapolating, and no-BC face classes.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.50`
- Reference identifier/location hits: `1/1`
- Latency: `22.53s`
- Matched reference signals: `bctypes.h, periodic, constant, zero, BcTypes.h`
- Missing/weak reference signals: `extrapolating, no-bc, face, classes`
- Claude Code answer excerpt: `The boundary condition (BC) types provided in `BcTypes.h` are defined in the `ippl::FieldBC` enum: - `PERIODIC_FACE` (0b0000) - `CONSTANT_FACE` (0b0001) - `ZERO_FACE` (0b0011) - `EXTRAPOLATE_FACE` (0b0100) - `NO_FACE` (0b1000) The header file also includes comments describing these as: 1. Periodic BC 2. Zero BC 3. S...`

bc_002 - How does a periodic boundary condition differ from a Neumann boundary condition in IPPL F

- Verdict: ****Partial****
- Correct reference answer: Periodic BCs wrap values from the opposite side of the domain; there is no explicit `NeumannFace` class in this codebase, so the closest non-periodic behavior is constant/extrapolating face treatment.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.41`
- Reference identifier/location hits: `0/1`
- Latency: `211.86s`
- Matched reference signals: `periodic, values, opposite, side, domain, there, behavior`
- Missing/weak reference signals: `bcs, wrap, explicit, neumannface, codebase, closest, non-periodic, constant/extrapolating`
- Claude Code answer excerpt: `In IPPL Fields, the difference between a periodic boundary condition and a Neumann boundary condition relates to how the system handles edges or boundaries of the domain: 1. ****Periodic Boundary Conditions:**** These are used to simulate an "infinite" or repeating domain by wrapping coordinates that exit one side of th...`

bc_003 - How are ghost cells populated during a halo exchange?

- Verdict: ****Partial****
- Correct reference answer: Ghost cells are populated mainly by `fillHalo()` / `PeriodicFace::apply()`: source slabs are packed, exchanged, and unpacked into ghost regions; `accumulateHalo\_noghost()` is a different operation.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.38`
- Reference identifier/location hits: `0/5`
- Latency: `27.59s`
- Matched reference signals: `ghost, cells, fillhalo, regions, operation`
- Missing/weak reference signals: `populated, mainly, periodicface::apply, source, slabs, packed, exchanged, unpacked`
- Claude Code answer excerpt: `The documentation for `HaloCells` in `data/raw/ippl/src/Field/HaloCells.hpp` describes the process for filling and accumulating ghost cells during a halo exchange. Specifically, there are two main methods for handling these operations: 1. **\*\*During an exchange (Inbound):\*\*** The method `accumulateHalo` calls `exchangeB...`

bc_004 - How are physical-cell values copied into ghost cells for periodic boundary conditions?

- Verdict: ****Partial****
- Correct reference answer: Periodic ghost cells are filled from opposite-side physical cells in `applyPeriodicSerialDim()` and the multi-rank `PeriodicFace::apply()` path.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.45`
- Reference identifier/location hits: `0/4`

- Latency: `161.01s`
- Matched reference signals: `periodic, ghost, cells, physical`
- Missing/weak reference signals: `filled, opposite-side, applyperiodicserialdim, multi-rank, periodicface::apply, path, PeriodicFace::apply, PeriodicFace::apply()`
- Claude Code answer excerpt: `periodic boundary conditions, physical-cell values are mapped to ghost cells (and vice versa) using coordinate transformation logic in the 'HaloCells' class within 'data/raw/ippl/src/Field/HaloCells.hpp'. Specifically, this is handled in the `**HaloCells::applyPeriodicSerialDim**` method (lines 234-291). When a dime...`

bc_005 - How does the absorbing boundary condition in the MaxwellSolvers module damp outgoing waves?

- Verdict: `**Incorrect**`
- Correct reference answer: The absorbing Maxwell BC uses second-order Mur-style formulas with precomputed weights and previous/current/next boundary values plus tangential neighbors to damp outgoing waves.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `51.12s`
- Missing/weak reference signals: `absorbing, maxwell, second-order, mur-style, formulas, precomputed, weights, previous/current/next`
- Claude Code answer excerpt: `! I'm your AI coding assistant. I can help you with a wide range of software engineering tasks, including: \* `**Code Generation:**` Writing new functions, classes, or entire modules based on your requirements. \* `**Debugging:**` Identifying and fixing bugs in your existing code. \* `**Refactoring:**` Improving the structur...`

Numerical Meaning

num_001 - What does the FFT-based Poisson solver physically compute in terms of Coulomb's law?

- Verdict: `**Partial**`
- Correct reference answer: The FFT Poisson solvers compute electrostatic potential (and optionally field/gradient) from charge density, i.e. the discretized Coulomb potential corresponding to $-\Delta\phi = \rho$.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.47`
- Reference identifier/location hits: `0/1`
- Latency: `39.11s`
- Matched reference signals: `poisson, solvers, electrostatic, potential, density, coulomb`
- Missing/weak reference signals: `fft, compute, optionally, field/gradient, charge, i.e, discretized, corresponding`
- Claude Code answer excerpt: `The FFT-based Poisson solver (specifically seen in the `FFTPeriodicPoissonSolver` and `FFTTruncatedGreenPeriodicPoissonSolver` implementations) computes the `**scalar potential**` ( $\phi$ ) from a given `**density field**` ( $\rho$ ). In terms of Coulomb's law, this relates to the calculation of the electrostatic potential...`

num_002 - What is the physical meaning of the fields advanced by the FDTD solver?

- Verdict: `**Correct**`
- Correct reference answer: The Maxwell solvers evolve electromagnetic field data: electric and magnetic fields (E , B) driven by a source/four-current field.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.64`
- Latency: `26.55s`
- Matched reference signals: `maxwell, electromagnetic, field, electric, magnetic, fields`
- Missing/weak reference signals: `solvers, evolve, driven, source/four-current`
- Claude Code answer excerpt: `The FDTD solver in the IPPL library evolves a 4-component vector field, denoted as  $A$ , which represents the combined state of the electromagnetic fields. Specifically, based on `FDTDSolverBase.hpp` (lines 116-137) and its usage in `StandardFDTDSolver.hpp`: \* `**Primary Vector (A):**` The solver tracks a 4-component...`

num_007 - What are the numerical tradeoffs between the FFT and CG Poisson solvers in IPPL?

- Verdict: `**Incorrect**`
- Correct reference answer: FFT Poisson is fast and direct on structured uniform grids with compatible BCs but less flexible; CG/PCG is iterative and may be slower/convergence-limited, but it is more flexible for alternative discretizations, BCs, and preconditioning.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.17`
- Latency: `31.32s`
- Matched reference signals: `fft, poisson, but`
- Missing/weak reference signals: `fast, direct, structured, uniform, grids, compatible, bcs, less`

- Claude Code answer excerpt: `I have investigated the `data/raw/ippl` repository for "FFT" and "Poisson" solvers but could not find any files that contain both terms together, nor was there a specific "Solver" directory or class found. It is possible the documentation regarding these tradeoffs is in the text files. Let me check the `.md` and `.t...`

Build And Install

build_001 - Which third-party libraries does IPPL depend on, and what are they used for?

- Verdict: ****Correct****
- Correct reference answer: Core third-party dependencies are MPI for distributed communication, Kokkos for performance portability, heFFTe for FFT backends/plans, FFTW/CuFFT as optional FFT backends, and GoogleTest for tests.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.50`
- Latency: `19.09s`
- Matched reference signals: `third-party, mpi, kokkos, heffte, fft, backends, googletest, tests`
- Missing/weak reference signals: `core, dependencies, distributed, communication, performance, portability, backends/plans, fftw/cufft`
- Claude Code answer excerpt: `The IPPL project depends on the following third-party libraries: \* **Kokkos** \*: Used as a primary foundation for high-performance computing. The project uses it to handle various backends (including CUDA and HIP) depending on the selected platform. It is integrated via `FetchContent` or found externally (lines 160-21...`

build_002 - What C++ standard does IPPL require, and why?

- Verdict: ****Partial****
- Correct reference answer: IPPL requires C++20, as stated in the README badge and the installation examples/configuration flags.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.38`
- Latency: `75.93s`
- Matched reference signals: `c++20, readme, flags`
- Missing/weak reference signals: `requires, stated, badge, installation, examples/configuration`
- Claude Code answer excerpt: `The current project is the IPPL (Independent Parallel Particle Layer) library. Based on my initial exploration: - **Project Goal:** It provides performance portable and dimension independent building blocks for scientific simulations using particle-mesh methods (multi-physics, various dimensions). - **Core Technolog...**`

build_003 - How is IPPL configured with CMake to enable a GPU build?

- Verdict: ****Partial****
- Correct reference answer: GPU builds are enabled through CMake options like `IPPL\_PLATFORMS=CUDA` or `IPPL\_PLATFORMS="HIP;OPENMP"` plus matching Kokkos/architecture/backend flags (`Kokkos\_ARCH\_\*`, `CMAKE\_CUDA\_ARCHITECTURES`, `Kokkos\_ENABLE\_HIP`, `Heffte\_ENABLE\_ROCM`, etc.).
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.45`
- Reference identifier/location hits: `2/6`
- Latency: `29.36s`
- Matched reference signals: `gpu, enabled, cmake, options, like, ippl\_platforms, cuda, openmp`
- Missing/weak reference signals: `builds, through, hip, matching, kokkos/architecture/backend, flags, kokkos\_arch\_, cmake\_cuda\_architectures`
- Claude Code answer excerpt: `Based on the `INSTALLATION.md` file, IPPL is configured using CMake with various options to enable features and support different hardware platforms. To specifically configure for a **GPU build**, the primary configuration parameters are: 1. **IPPL\_PLATFORMS**: This option determines the supported architectures. F...`

build_004 - What options does INSTALLATION.md describe for configuring Kokkos for IPPL?

- Verdict: ****Partial****
- Correct reference answer: `INSTALLATION.md` describes Kokkos version selection plus backend/architecture options used with IPPL builds, including CUDA and HIP/OpenMP examples and `IPPL\_PLATFORMS` combinations.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.15`
- Reference identifier/location hits: `1/2`
- Latency: `4.59s`
- Matched reference signals: `installation.md, kokkos, INSTALLATION.md`
- Missing/weak reference signals: `describes, version, selection, backend/architecture, options, builds, cuda, hip/openmp`

- Claude Code answer excerpt: `find information about configuring Kokkos for IPPL, I will check the `data/raw/ippl` directory for an `INSTALLATION.md` file.`

build_005 - How do I install IPPL on the EULER cluster according to the documentation?

- Verdict: ****Correct****
- Correct reference answer: The EULER install guide says to connect with VPN/SSH, switch to the new software stack, purge modules, load GCC/CMake/CUDA/OpenMPI, clone the repo, configure a build, compile, and test on an interactive node.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.55`
- Latency: `24.59s`
- Matched reference signals: `euler, install, connect, new, software, stack, purge, load`
- Missing/weak reference signals: `guide, says, vpn/ssh, switch, modules, gcc/cmake/cuda/openmpi, repo, configure`
- Claude Code answer excerpt: `To install IPPL on the EULER cluster, follow these steps as outlined in `data/raw/ippl/doc/extras/IPPLonEULER.md`: 1. **\*\*Connect to the Cluster\*\***: Use SSH with X11 forwarding enabled: `ssh -Y <username>@euler.ethz.ch` 2. **\*\*Prepare the Environment\*\***: \* Transition to the new software stack: `env2lmod` \* Clean the envir...`

Testing And Workflow

test_002 - What naming conventions does WORKFLOW.md require for variables, constants, and member v

- Verdict: ****Correct****
- Correct reference answer: `WORKFLOW.md` requires camelCase variables, CAPITAL_CASE compile-time constants, and member names suffixed with `\_m`.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.50`
- Reference identifier/location hits: `2/2`
- Latency: `11.38s`
- Matched reference signals: `workflow.md, variables, constants, member, suffixed, WORKFLOW.md, \_m`
- Missing/weak reference signals: `requires, camelcase, capital\_case, compile-time, names`
- Claude Code answer excerpt: `Based on the `WORKFLOW.md` file, the naming conventions for variables, constants, and member variables are as follows: - **\*\*Variables\*\***: Should use camel casing (line 3). - **\*\*Constants\*\***: Compile time constants should use capital casing (line 4). - **\*\*Member variables\*\***: Should be suffixed with `_m` (line 5).`

test_003 - When should code use Kokkos::sqrt versus std::sqrt according to WORKFLOW.md?

- Verdict: ****Correct****
- Correct reference answer: Use `std::sqrt` in host-only code, and `Kokkos::sqrt` in device-capable code such as `KOKKOS\_INLINE\_FUNCTION` kernels.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.62`
- Reference identifier/location hits: `3/3`
- Latency: `15.47s`
- Matched reference signals: `std::sqrt, host-only, kokkos::sqrt, such, kokkos\_inline\_function, KOKKOS\_INLINE\_FUNCTION, Kokkos::sqrt`
- Missing/weak reference signals: `use, device-capable, kernels`
- Claude Code answer excerpt: `According to `WORKFLOW.md`, the choice between `Kokkos::sqrt` and `std::sqrt` depends on the execution context: - **\*\*`std::sqrt`\*\***: Should be used for any instance of a mathematical function in **\*\*host-only code\*\***. - **\*\*`Kokkos::sqrt`\*\***: Should be used for instances occurring in **\*\*device-code\*\*** or **\*\*code that might be...`**

test_004 - How does IPPL profile MPI calls using mpiP?

- Verdict: ****Correct****
- Correct reference answer: IPPL profiles MPI calls with mpiP by preloading `libmpiP.so`, optionally setting `MPIP` flags, then parsing the produced `.mpiP` output with the bundled `scripts/mpiP.py`.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.41`
- Reference identifier/location hits: `4/5`
- Latency: `18.57s`
- Matched reference signals: `mpi, mpiP, then, parsing, output, MPIP, mpiP, mpiP.py`
- Missing/weak reference signals: `profiles, calls, preloading, libmpip.so, optionally, setting, flags, produced`
- Claude Code answer excerpt: `IPPL uses the `mpiP` tool (implemented as a Python script in `data/raw/ippl/scripts/mpiP.py`) to process and aggregate data from `.mpiP` files, which typically contain

information about MPI communication events. The `mpiP.py` script works by: 1. Taking a directory path as an argument. 2. Searching for any file in th...`

test_005 - What does the SLURM job template for Gwendolen in the README look like?

- Verdict: ****Partial****
- Correct reference answer: The README does include a Gwendolen SLURM template showing the `gmerlin6` cluster, `gwendolen` partition, GPU count, and `srun ./<your\_executable> <args> --kokkos-map-device-id-by=mpi\_rank`.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.44`
- Reference identifier/location hits: `1/3`
- Latency: `36.60s`
- Matched reference signals: `readme, gwendolen, slurm, template, partition, gpu`
- Missing/weak reference signals: `showing, gmerlin6, cluster, count, srun, your\_executable, args, kokkos-map-device-id-by`
- Claude Code answer excerpt: `SLURM job template for Gwendolen is not explicitly displayed as a full script in the `README.md` file. However, the **\*\*Running the system\*\*** section of the README (referenced via the project instructions) mentions that `sbatch job.sh` is used as the "SLURM launcher for the gwendolen GPU partition."`